

StaleyNET: Text-Conditioned Music Generation Trained from Scratch on Edge GPUs in Rust

Arpan Pathak

November 2026

Abstract

Text-to-music generators are assembled from large pretrained parts: a neural codec, a text encoder and a language model trained on tens of thousands of hours of audio. We ask what can be learned when no such part is available. StaleyNET is written entirely in Rust, GPU kernels included, and trained from random initialisation on embedded GPUs with 23 hours of public music. It describes sound as residual k-means tokens of the log-mel spectrogram and predicts them with a Transformer conditioned on the bytes of a caption and, optionally, of lyrics. Because every stage is our own, every stage can be measured alone. Rendering audio through each stage shows that the tokenizer, not the language model, decides how the output sounds: the tokenizer causes 3.6 times the spectral error of the vocoder, while a model that memorises an excerpt reproduces it token for token. We use this measurement to guide the design of finer tokens and larger models, and we introduce a paired test that determines whether a model actually reads the lyrics it is given.

1 Introduction

Music generation from text has improved rapidly [1, 3]. The leading systems share a recipe: a pretrained neural codec turns audio into tokens, a pretrained text encoder turns the prompt into vectors, and a large Transformer connects the two. Each part is trained separately on large data, and the finished system reveals little about which part is responsible for what one hears.

We build every part ourselves. StaleyNET (Figure 1) computes its own spectrograms, learns its own tokenizer, trains its own Transformer and reconstructs its own waveforms, all in Rust, with the GPU kernels compiled from Rust by cuda-oxide [19]. It trains on NVIDIA Jetson devices, GPUs built for robots and cars. Because each stage is our own, each can be measured in isolation, and the stage that limits the result can be found and improved.

The measurements lead to a clear diagnosis. The model learns text-conditioned structure well above chance, yet the voices it produces are blurred. Two experiments locate the cause. We pass held-out recordings through the tokenizer and vocoder without any model, and find that the tokenizer alone removes most of the spectral detail. We then train the model to memorise a single excerpt, and find that it reproduces the excerpt exactly. The model is capable; the tokens are too coarse.

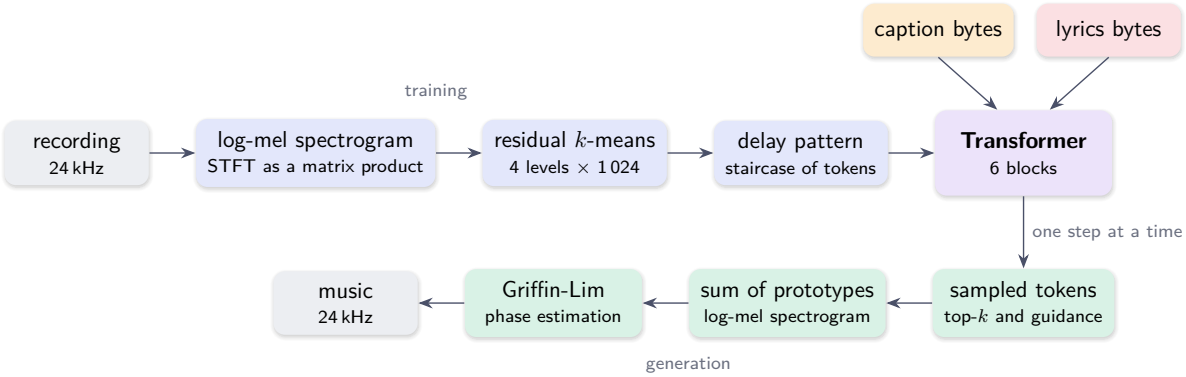


Figure 1: StaleyNET. In training (top row), a recording becomes a staircase of discrete tokens that the Transformer learns to predict from the caption and lyrics. In generation (bottom row), the Transformer produces tokens one step at a time, and the tokens are turned back into sound.

The paper makes 4 contributions.

- A text-to-music pipeline written entirely in Rust, GPU kernels included, and trained from scratch on edge GPUs.
- A presentation of each algorithm with a worked example and, where an algorithm is easy to get wrong, a proof of correctness.
- A position map that adds a lyrics input to a trained model without discarding what it learned, and a paired test of whether the input is used.
- A stage-by-stage analysis that separates the error of the tokenizer, the vocoder and the language model.

We write algorithms in the style of Cormen et al. [4], as in our tutorial on tiled matrix multiplication [20]: procedure names in capitals, numbered lines, and mathematical notation inside the pseudocode. The range $[a, b)$ denotes the integers $a, \ldots, b-1$, indices start at 0, and losses are in nats.

2 Related work

AudioLM [2] showed that audio can be generated by a language model over discrete tokens. MusicLM [1] conditioned this on text, and MusicGen [3] reduced the design to one Transformer over the codebooks of a neural codec, arranged by a delay pattern. We keep MusicGen's delay pattern, top-k sampling [9] and classifier-free guidance [12], and replace its pretrained parts with k-means over log-mel frames and with raw text bytes [31].

Jukebox [7] generated singing from lyrics and learned the alignment of words and sound from 1.2 million songs, while singing synthesis systems such as DiffSinger [16] require a score with phoneme durations. Our lyrics experiments use the line-level timings of JamendoLyrics [27, 8]. Neural codecs [32, 6, 15] and neural vocoders [14, 24] reach far higher fidelity than k-means tokens and Griffin-Lim [10, 21], at a training cost far beyond our setting; Section 9.1 measures what our choice costs.

3 From sound to spectrogram and back

A waveform at 24 kHz holds 24 000 numbers per second, most of which describe the phase of each vibration, which the ear barely hears. We describe sound instead by its log-mel spectrogram: the energy in 128 bands of perceptual frequency, measured 94 times per second, on a logarithmic scale.

3.1 Resampling

Recordings arrive at various rates, mostly 44.1 kHz, and are brought to 24 kHz. Lowering the rate requires removing all content above the new Nyquist frequency of 12 kHz first, or it folds back into the audible range as aliasing.

Definition 3.1 (Bandlimited resampling)**.** Given samples x_m at rate f_{in}, the resampled signal at rate f_{out} is

$$y_n \;=\; \sum_{m} x_m \, h(t_n - m)\,, \qquad t_n = n\,\frac{f_{\text{in}}}{f_{\text{out}}},$$

where h is a low-pass filter. Output sample n is a weighted sum of the input samples near the fractional position t_n.

We use a windowed sinc, $h(t) = c\,\operatorname{sinc}(ct)\,w(ct/Z)$, with cutoff c relative to the input rate, $Z = 24$ zero crossings on each side and a Kaiser window w with $\beta = 8.6$ [26]. Evaluating h anew for every output sample would be slow, but the fractional part of t_n takes few values.

Worked example. For 44 100 Hz to 24 000 Hz the rates share the factor 300, so $t_n = 147n/80$. For $n = 1$, $t_1 = 1 + 67/80$. Every t_n has a fraction $p/80$ with $p \in [0, 80)$, so exactly 80 sets of weights, the *phases* of a polyphase filter, occur. We compute them once.

RESAMPLE$(x, f_{\text{in}}, f_{\text{out}})$

```
1   g = gcd(f_in, f_out)
2   L = f_out / g   // number of phases
3   M = f_in / g    // input step per output sample
4   c = 0.97 · min(1, f_out / f_in)
5   R = ⌈Z / c⌉
6   for p ∈ [0, L) {
7       for j ∈ [−R, R] {
8           h_p[j] = c · sinc(c (j − p/L)) · w(c (j − p/L) / Z)
9       }
10  }
11  for each output index n {
12      m = ⌊nM / L⌋   // whole part of t_n
13      p = nM mod L   // fraction of t_n, in L-ths
14      y_n = Σ_{j∈[−R,R]} h_p[j] · x_{m+j}
15  }
```

Lines 1–5 fix the geometry, lines 6–10 build the L phases once, and line 14 is the single dot product per output sample. The factor 0.97 in line 4 keeps the transition band of the filter

below the new Nyquist frequency. A test confirms the effect: a 15 kHz tone is suppressed by more than 80 dB when 48 kHz audio is converted to 24 kHz.

3.2 The spectrogram as a matrix product

The signal is cut into frames of $N = 1024$ samples, $H = 256$ apart, and each frame is multiplied by a Hann window. The strength of every frequency in every frame is usually computed with the fast Fourier transform. We compute it with a matrix product, so that the spectrogram runs on the same GPU kernel as the Transformer and inherits its tests (Section 8).

Definition 3.2 (Spectrogram)**.** Let row f of F hold frame f, and let W have entries $W_{nk} = w[n]\, e^{-2\pi i k n / N}$ for $k \in [0, N/2]$. The log-mel spectrogram is

$$S \;=\; \ln \max\bigl(|FW|^2 B,\; 10^{-5}\bigr),$$

where the square is taken entry by entry and B holds 128 triangular mel filters [25].

Column k of W is a windowed sine and cosine at $k \cdot 23.4\,\mathrm{Hz}$, so row f of FW measures how much of each frequency frame f contains. The real and imaginary parts of W are 2 real matrices, and B adds a third product. The method needs about $N / \log_2 N \approx 100$ times the arithmetic of an FFT, yet on the Orin NX it analyses 60 s of audio in 78 ms, against 98 ms for a multithreaded CPU FFT, and agrees with it within 10^{-2} log units.

3.3 Estimating the phase

A generated spectrogram holds the strength of every frequency but not its phase. Griffin-Lim [10] recovers a phase by alternating between 2 requirements: the magnitudes must match the target, and the spectrogram must belong to an actual waveform.

GRIFFIN-LIM$(\,|S|, T\,)$

1. $X = |S| \cdot e^{i\phi}$ with random phases ϕ
2. $x = \mathrm{ISTFT}(X)$
3. $P_{\mathrm{prev}} = \mathrm{STFT}(x)$
4. **for** $t \in [0, T)$ {
5. $\quad P = \mathrm{STFT}(x)$ *// nearest consistent spectrogram*
6. $\quad A = P + \alpha\,(P - P_{\mathrm{prev}})$ *// momentum*
7. $\quad X = |S| \cdot A / |A|$ *// keep phase, restore magnitude*
8. $\quad P_{\mathrm{prev}} = P$
9. $\quad x = \mathrm{ISTFT}(X)$
10. }
11. **return** x

Line 5 replaces the estimate by the spectrogram of a real waveform, and line 7 restores the target magnitudes. Line 6 is the acceleration of Perraudin et al. [21], which extrapolates the last change with $\alpha = 0.99$; in the first round it has no effect, since $P = P_{\mathrm{prev}}$. We use $T = 64$, after mapping the 128 mel bands back to 513 frequencies with a regularised least-squares inverse of B.

4 From spectrogram to tokens

A language model chooses each next symbol from a finite vocabulary, but a spectrogram is made of real numbers. The tokenizer bridges the two. After each band is standardised with its training-set mean and deviation, 2 consecutive frames form a *step*: 256 numbers covering 21.3 ms.

4.1 Residual quantisation

The idea is that of describing a colour first by its family, then by its shade within the family, then by finer and finer corrections. Each level names the nearest of 1 024 prototypes to what the previous levels left unexplained.

Definition 4.1 (Residual vector quantisation)**.** Given C codebooks $E_0, \dots, E_{C-1}$ of K prototypes each, a step x is encoded as indices $c_0, \dots, c_{C-1}$ by

$$r_0 = x, \qquad c_\ell = \arg\min_{k \in [0,K)} \|r_\ell - E_\ell[k]\|^2,$$
$$r_{\ell+1} = r_\ell - E_\ell[c_\ell],$$

and decoded as $\hat{x} = \sum_\ell E_\ell[c_\ell]$. We use $C = 4$ and $K = 1024$.

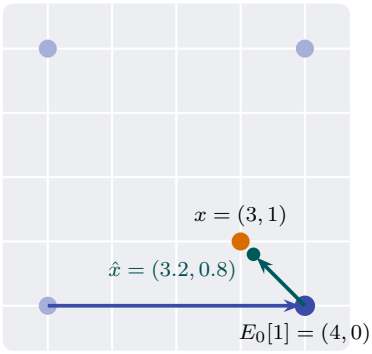


Figure 2: Residual quantisation of $x = (3, 1)$ in 2 levels. Level 0 (blue) chooses the prototype $(4, 0)$; level 1 (green) adds the correction $(-0.8, 0.8)$.

Worked example. In 2 dimensions, take the codebooks

$$E_0 = \{(0,0),\ (4,0),\ (0,4),\ (4,4)\},$$
$$E_1 = \{(0.8,0.8),\ (-0.8,0.8),\ (-0.8,-0.8),\ (0.8,-0.8)\},$$

and encode $x = (3, 1)$ (Figure 2). The squared distances to E_0 are $10, 2, 18, 10$, so $c_0 = 1$ and $r_1 = (-1, 1)$. The nearest entry of E_1 is $(-0.8, 0.8)$, so $c_1 = 1$ and $r_2 = (-0.2, 0.2)$. The error falls from $\sqrt{2} \approx 1.41$ after 1 level to $\sqrt{0.08} \approx 0.28$ after 2.

Each step thus becomes 4 integers in $[0, 1024)$, ordered from coarse to fine: 40 bits per 21.3 ms, or 1.9 kbit/s.

4.2 Training the codebooks

TRAIN-CODEBOOKS(X, C, K, I)

```
1   R = X // what remains to be described
2   for ℓ ∈ [0, C) {
3     E_ℓ = K rows of R, chosen at random
4     repeat I times {
5       a_i = NEAREST(R_i, E_ℓ) for each row i
6       E_ℓ[k] = mean{R_i : a_i = k} for each k
7       if no row has a_i = k
8         E_ℓ[k] = a random row of R
9     }
10    R_i = R_i − E_ℓ[NEAREST(R_i, E_ℓ)] for each row i
11  }
```

Lines 5–6 are Lloyd's 2 moves [17]: assign, then average. Lines 7–8 revive a prototype that attracted nothing, so that none of the 1 024 entries is wasted, and line 10 hands the residuals to the next level. We run $I = 20$ rounds on 200 000 steps drawn from the training set.

4.3 The nearest prototype as a matrix product

Lemma 4.1. $\arg\min_k \|x - e_k\|^2 = \arg\min_k \left(\|e_k\|^2 - 2\, x \cdot e_k\right)$.

Proof. Expand $\|x - e_k\|^2 = \|x\|^2 - 2\, x \cdot e_k + \|e_k\|^2$; the first term does not depend on k. □

In the example, the norms $\|e_k\|^2$ are $0, 16, 16, 32$ and the dot products $x \cdot e_k$ are $0, 12, 4, 16$, giving scores $0, -8, 8, 0$; the minimum again picks $(4, 0)$. On the GPU, the dot products of 8 192 steps with 1 024 prototypes form one 8192×256 by 256×1024 matrix product, and a small kernel takes the minimum of each row.

5 The model

The model (Figure 3) is a decoder-only Transformer [28, 23] that reads text and audio as one sequence and predicts, at each position, what comes next, in the way a text model predicts the next word.

5.1 One sequence for text and audio

The vocabulary holds the 256 byte values [31], 3 marks (`BEGIN`, `SEPARATOR`, `PADDING`) and $C \cdot K = 4096$ audio symbols. The text fills a fixed 128 positions, `BEGIN`, 126 caption bytes and `SEPARATOR`, so that audio always begins at the same position. An audio position carries all 4 integers of a step, and its input vector is the sum of their embeddings and the embedding of the position.

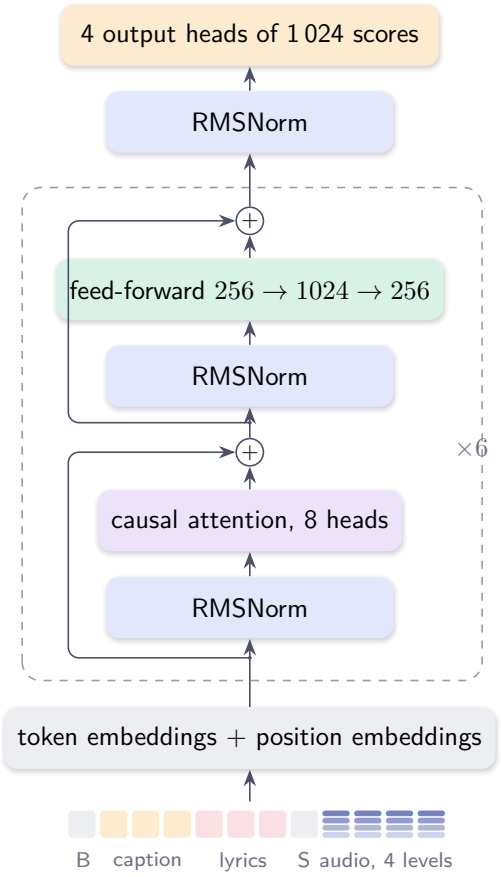


Figure 3: The model. Text bytes and audio tokens share one sequence; each audio position carries the 4 levels of one step. Six pre-normalised blocks, each adding its result to its input, end in 4 output heads, one per level.

5.2 The delay pattern

The 4 integers of a step are predicted from one position, yet a fine level corrects a coarse one and is best predicted once the coarse one is known.

Definition 5.1 (Delay pattern [3])**.** Position t of the delayed sequence holds, in slot ℓ, the integer $a^{\ell}_{t-\ell}$ of level ℓ for frame $t-\ell$, and an empty mark where $t-\ell$ is not a frame. F frames become $F+C-1$ positions.

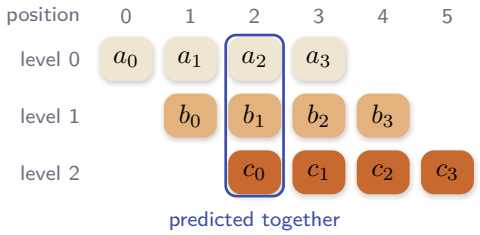


Figure 4: The delay pattern for 3 levels and 4 frames. At position 2 the model predicts a_2, b_1 and c_0; the coarser integers they refine are already in its input.

In Figure 4, position 2 predicts a_2, b_1 and c_0 together, while a_1, which b_1 refines, and a_0, b_0, which c_0 refines, are already known. The cost is $C-1=3$ extra positions, against a fourfold longer sequence if each level had its own positions.

5.3 The network and its objective

Each of the 6 blocks lets every position gather information from itself and earlier positions through attention, then transforms each position with a small feed-forward network; both parts normalise their input [33, 30] and add their output to it:

$$h = x + \mathrm{Attention}(\mathrm{Norm}(x)),$$
$$y = h + \mathrm{FeedForward}(\mathrm{Norm}(h)).$$

The width is 256, the 8 heads have width 32, and the feed-forward network uses GELU [11]. The model has 7.1 million parameters.

At position p the model outputs, for each level ℓ, a distribution over the integer $y_{p,\ell}$ that appears at position $p+1$. The loss is the cross-entropy averaged over the positions V_ℓ that have such a target, and then over the levels:

$$\mathcal{L} = \frac{1}{C}\sum_{\ell=0}^{C-1}\frac{1}{|V_\ell|}\sum_{p\in V_\ell} -\ln P_\theta(y_{p,\ell} \mid z_0,\dots,z_p),$$

where z_p is the input at position p. A uniform guess scores $\ln 1024 \approx 6.93$.

5.4 Gradients by hand

No automatic differentiation exists for Rust on the GPU, so each layer implements its own gradient. Attention is the least obvious case. For one head with queries Q, keys K, values V and width $d=32$,

$$S = QK^{\top}/\sqrt{d} + M, \qquad P = \mathrm{softmax}(S), \qquad O = PV,$$

where M is $-\infty$ above the diagonal and the softmax acts on rows.

Proposition 5.1. *Given ∂O, the gradients are $\partial V = P^\top \partial O$, $\partial P = \partial O\, V^\top$, $\partial S_{ik} = P_{ik}(\partial P_{ik} - \rho_i)$ with $\rho_i = \sum_j \partial P_{ij} P_{ij}$, $\partial Q = \partial S\, K/\sqrt{d}$ and $\partial K = \partial S^\top Q/\sqrt{d}$.*

Proof. The first 2 follow from $O = PV$. Within a row, $\partial P_{ij}/\partial S_{ik} = P_{ij}(\delta_{jk} - P_{ik})$, so $\partial S_{ik} = \sum_j \partial P_{ij} P_{ij}(\delta_{jk} - P_{ik}) = P_{ik}(\partial P_{ik} - \rho_i)$. The last 2 follow from $S = QK^\top/\sqrt{d}$; where $M = -\infty$, $P_{ik} = 0$ and so $\partial S_{ik} = 0$. □

Apart from ∂S, every gradient is a matrix product and runs on the same kernel as the forward pass. A test perturbs 3 random entries of every parameter tensor by $\pm 10^{-2}$ and requires the measured change of the loss to agree with the computed gradient within 5 %.

6 Lyrics

A caption says how a piece should sound, not which words to sing. Lyrics need their own input, and training examples in which the model can hear which words are sung where.

6.1 A position map that preserves a trained model

The lyrics take $L = 256$ byte positions between the caption and the separator. Each position p reads its position embedding from row $\pi(p)$ of a table, and the choice of π decides whether a model trained without lyrics can be extended (Figure 5).

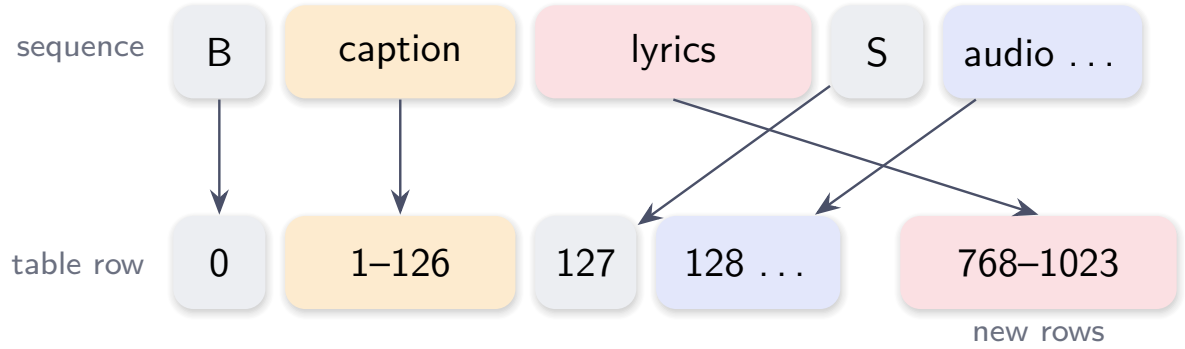


Figure 5: The position map π. Positions outside the lyrics keep the rows they used before lyrics were added; the lyrics use new rows appended to the table.

Definition 6.1 (Position map)**.** With the caption at positions $[1, 1{+}P)$, the lyrics at $[1{+}P, 1{+}P{+}L)$, and T rows in the model without lyrics,

$$\pi(p) = \begin{cases} p & \text{if } p < 1 + P, \\ T + p - 1 - P & \text{if } 1 + P \le p < 1 + P + L, \\ p - L & \text{otherwise.} \end{cases}$$

Lemma 6.1. *Every position outside the lyrics uses the row that the same part of the sequence used without lyrics, and the lyrics use only the new rows $[T, T + L)$.*

Proof. Caption positions are unchanged. The separator and the audio move right by L, which π subtracts. The lyrics map onto $T + [0, L)$. □

With $P = 2$, $L = 3$ and $T = 16$, positions $0, \ldots, 8$ use rows $0, 1, 2, 16, 17, 18, 3, 4, 5$. A trained model is therefore extended by appending L new rows to its table; every learned parameter keeps its meaning, and training continues from there.

6.2 Pairing audio with the words sung in it

A training example covers 13.6 s of a song and must carry the lines sung in that window. JamendoLyrics [8] gives the start and end of every line of 78 songs in English, French, German and Spanish; a window receives each line that overlaps it. Takes recorded with our studio carry the text without timings, and their lines are assumed to divide the take evenly, a fair approximation for short takes of continuous singing. Since the lyrics corpus is small (4.7 h against 23 h), each dataset has a sampling weight: weight w makes each of its songs w times as likely to be drawn.

6.3 Does the model read the lyrics?

A model can lower its loss on the lyrics corpus by learning how those songs sound, without reading their words. We therefore test the use of the lyrics directly, by giving the model the same audio with the right words, with no words and with someone else's words.

LYRICS-TEST$(\theta, \mathcal{D}, B)$

1 **for** $b \in [0, B)$ {
2 draw a batch of windows from held-out songs $\mathcal{D}$
3 $\mathcal{L}_b^{\text{right}}$ = loss of θ with each window's own lyrics
4 $\mathcal{L}_b^{\text{none}}$ = loss of θ with empty lyrics
5 $\mathcal{L}_b^{\text{wrong}}$ = loss of θ with lines from another song
6 }
7 $\Delta_b = \mathcal{L}_b^{\text{wrong}} - \mathcal{L}_b^{\text{right}}$ for each b
8 $\bar{\Delta} = \frac{1}{B} \sum_b \Delta_b$
9 $s = \sqrt{\text{Var}(\Delta)/B}$
10 **return** $\bar{\Delta}$, s, and whether $\bar{\Delta} > 2s$

Lines 3–5 score the same windows, so line 7 compares like with like and removes the variation between windows. A model that ignores the words gives $\bar{\Delta} \approx 0$; one that reads them predicts the audio better with the right words, so $\bar{\Delta} > 0$. The wrong lines in line 5 always begin at a sung line, never at silence, and comparing lines 4 and 5 separates a reaction to the presence of text from a reaction to its content.

7 Generation

Generation repeats one step: predict a distribution for each level at the next position, draw an integer from each, and append them.

7.1 The key/value cache

Position p attends to the keys and values of positions $0, \ldots, p$. These never change once computed, because no position attends forward, so a cache keeps them [22].

DECODE-STEP(z, p, cache)

1 $h = \text{embed}(z) + \text{position}(\pi(p))$
2 **for** each block j {
3 $\quad q, k, v =$ projections of $\text{Norm}(h)$
4 $\quad \mathit{cache}_j[p] = (k, v)$
5 $\quad h = h + \text{Attend}(q,\ \mathit{cache}_j[0\,..\,p])$
6 $\quad h = h + \text{FeedForward}(\text{Norm}(h))$
7 }
8 **return** the 4 groups of scores of $\text{Norm}(h)$

Theorem 7.1. *Feeding a sequence position by position through* ***DECODE-STEP*** *gives, at every position, the scores of a single forward pass over the whole sequence.*

Proof. By induction over the blocks. At block 0 the input at p depends only on the token and position of p. Suppose the inputs of block j agree up to position p. Then the keys and values of line 3 agree, and when p is processed the cache holds exactly those of positions $0, \ldots, p$. In the full pass the mask gives zero weight to later positions, so attention at p combines the same keys and values with the same weights, and the feed-forward network acts on each position alone. The outputs of block j therefore agree up to p. □

A test confirms the theorem within 10^{-4} at every position, for models with and without lyrics, which also checks π. The cache has a fixed capacity, so a step allocates no memory and costs 3.2 ms instead of 23.4 ms for recomputing the context.

7.2 Sampling and guidance

We keep the $k = 250$ most probable integers of each level and draw among them [9]. Because the text was removed from 10 % of the training examples, the model predicts both with the text, z^{text}, and without it, z^{none}. Classifier-free guidance [12] amplifies their difference:

$$z = z^{\text{none}} + s\,(z^{\text{text}} - z^{\text{none}}).$$

If 2 integers score $(2, 1)$ with the text and $(1, 1)$ without, the first has probability 0.73; with $s = 3$ the scores become $(4, 1)$ and its probability 0.95. We use $s = 3$. Lyrics longer than L bytes are cut at line breaks into windows that are fed in turn, each over an equal share of the output, with the recent audio kept as context.

8 GPU kernels

Matrix products carry nearly all of the arithmetic: the spectrogram, the nearest-prototype search, the projections of the Transformer and both products of attention. Our tutorial [20] derives the shared-memory tiled product; here we take it one step further and show how one gradient is made repeatable.

8.1 Register blocking

In the tiled kernel a block of 16×16 threads copies tiles of A and B into shared memory, which cuts global traffic 16-fold [20, 13]. Each multiply-add still reads 2 values from shared memory, which is slower than a thread's registers. Register blocking [29] lets each thread compute an $r \times r$ group of outputs and reuse every value it reads r times (Figure 6).

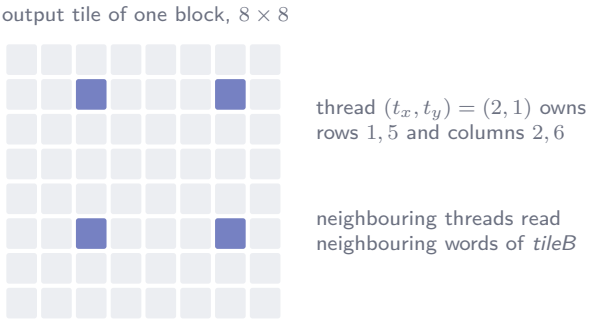


Figure 6: Register blocking with 4×4 threads and $r = 2$. Each thread computes r^2 outputs spread over the tile, rows $t_y + 4i$ and columns $t_x + 4j$. Our kernel uses 16×16 threads, $r = 4$ and a 64×64 tile.

BLOCKED-THREAD(t_x, t_y) tile 64×64, depth 16, $r = 4$

```
1   sum[i][j] = 0 for i, j ∈ [0, r)
2   for each phase t {
3     copy a 64 × 16 tile of A into tileA
4     copy a 16 × 64 tile of B into tileB
5     BARRIER()
6     for u ∈ [0, 16) {
7       α_i = tileA[t_y + 16i][u] for i ∈ [0, r)
8       β_j = tileB[u][t_x + 16j] for j ∈ [0, r)
9       sum[i][j] = sum[i][j] + α_i β_j for i, j ∈ [0, r)
10    }
11    BARRIER()
12  }
13  store sum[i][j] at row t_y + 16i, column t_x + 16j of C
```

Lines 7–8 read $2r = 8$ values and line 9 performs $r^2 = 16$ multiply-adds, so each multiply-add costs $2/r = 0.5$ shared-memory reads instead of 2. Global traffic also falls: each phase copies $2 \cdot 64 \cdot 16$ values for $64 \cdot 64 \cdot 16$ multiply-adds, so a product of an $M \times K$ and a $K \times N$ matrix fetches

$$\frac{M}{64} \cdot \frac{N}{64} \cdot \frac{K}{16} \cdot 2 \cdot 64 \cdot 16 = \frac{2MNK}{64}$$

values, a quarter of the $2MNK/16$ of 16×16 tiles. The 2 barriers play the roles proved correct for the tiled kernel [20]. On the Thor, the blocked kernel cut a training step on 4×768 positions from 238 ms to 147 ms.

8.2 A repeatable embedding gradient

The embedding gradient adds the gradient g_p of each position into the row of its token z_p. Atomic additions would let threads add into a shared row in a varying order, and floating-point addition is not associative, so the result would vary between runs. We fix the order with a counting sort.

EMBEDDING-GRADIENT(z, g, V)

```
count[v] = |{p : z_p = v}| for v ∈ [0, V)  // CPU
start[v] = Σ_{u<v} count[u]
list = positions sorted by token, then by position
for each token v and column c, in parallel {  // GPU
    G[v][c] = Σ g_p[c] over p in list[start[v] .. start[v + 1]), in order
}
```

For tokens $z = (2, 0, 2, 1)$, the counts are $(1, 1, 2)$, the starts $(0, 1, 2, 4)$ and the list $(1, 3, 0, 2)$, so token 2 receives $g_0 + g_2$ in that order. Each entry is summed by 1 thread in a fixed order, so the result is the same on every run, and a whole training run repeats bit for bit.

9 Experiments

We used a Jetson Orin NX 16 GB with CUDA 12.6 and a Jetson AGX Thor. The main corpus is the Song Describer Dataset [18], 706 two-minute Creative Commons excerpts with 1 106 captions written by listeners. Songs are divided by artist, so that no voice appears on both sides: 667 for training and 39 held out. Spectral errors are mean absolute differences of natural-log mel energies, 1 unit being about 8.7 dB.

Correctness and speed. The 38 tests pass on the Orin NX, and the kernel tests also on the Thor. They compare every kernel with a CPU implementation at awkward sizes, check every gradient numerically, verify the cache theorem, and confirm that a small network memorises a fixed batch. Table 1 shows that tiling speeds up the matrix product 5.8 to 8.2 times on the Orin NX; a training step on 4×512 positions falls from 2 919 ms to 537 ms.

Table 1: Matrix products on the Orin NX, in GFLOP/s.

product ($M \times N \times K$)	direct	tiled	speed-up
attention projections ($2048 \times 768 \times 256$)	32.2	263.4	8.2
feed-forward ($2048 \times 1024 \times 256$)	42.0	264.2	6.3
output heads ($2048 \times 4096 \times 256$)	30.5	217.4	7.1
spectrogram ($4096 \times 513 \times 1024$)	38.6	224.0	5.8

Training. A first run of 1 500 steps reached a held-out loss of 4.93, and a run of 15 000 steps on the Thor 4.02 (Table 2). Restricted to the 281 songs whose captions mention a voice, a model fine-tuned from the long run peaked after 250 steps at 4.23, after which its held-out loss rose while its training loss fell; a model trained from scratch on the same songs reached 4.72. The vocal subset is too small to train on for long.

Table 2: Best held-out loss, in nats per token (uniform guess 6.93).

run	data	steps	loss
first run, Orin	all songs	1 500	4.93
long run, Thor	all songs	15 000	4.02
fine-tuned, Thor	vocal songs	250	4.23
from scratch, Orin	vocal songs	5 500	4.72

9.1 What limits the sound

Generated voices sound pitched but blurred. To find the cause without involving the model, we rendered 12 s excerpts of 4 held-out vocal songs through each stage and measured the error against their exact spectrograms (Figure 7 and Table 3).

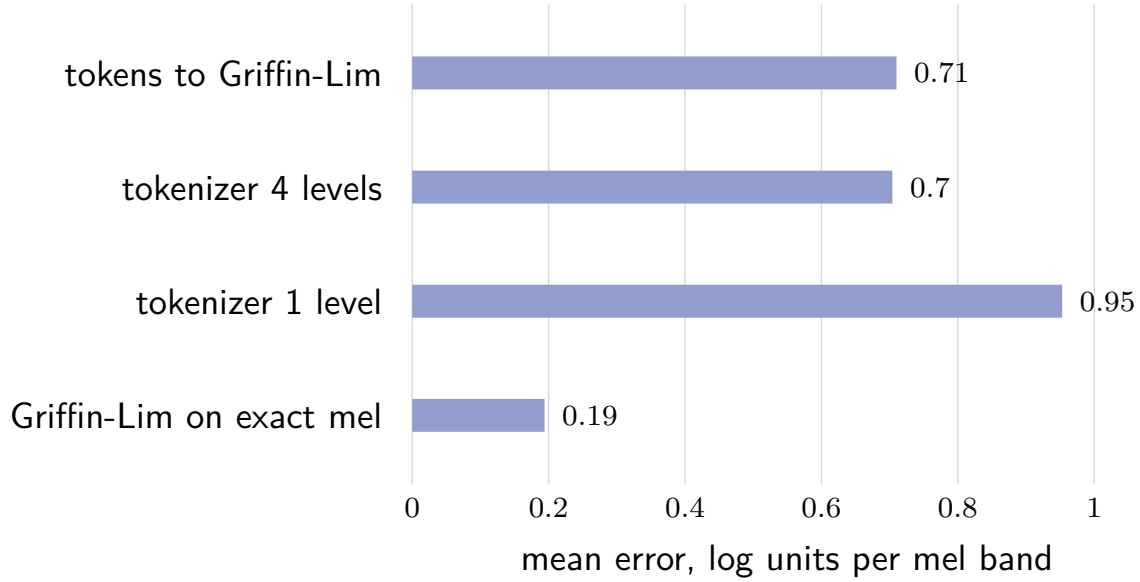


Figure 7: Where the spectral detail is lost. The tokenizer causes 3.6 times the error of the vocoder, and a perfect model (bottom bar) is limited almost entirely by the tokenizer.

Table 3: Spectral error of each stage on 4 held-out vocal excerpts.

excerpt	vocoder	1 level	4 levels	end to end
guitar, brass, female voice	0.230	1.055	0.807	0.821
folk metal, male voice	0.152	0.973	0.707	0.703
electronic, vocal sample	0.170	0.801	0.569	0.554
bass, piano, wordless voice	0.222	0.984	0.732	0.763
mean	0.194	0.953	0.704	0.710

An error of 0.7 log units, about 6 dB per band, erases the harmonic and formant detail that makes a voice intelligible. The measure ignores the phase artefacts of Griffin-Lim, which remain audible; the tokenizer is the main spectral limit, not the only audible one.

To test the model alone, we trained it on a single 12 s excerpt of 562 frames. Its loss fell to 0.054 after 100 steps and 0.002 after 200. Generating 12 s from the caption and comparing the integers with the excerpt's (Table 4), greedy decoding reproduces all 562 frames, and top-k sampling departs only at isolated integers without drifting.

Table 4: Generated integers against the memorised excerpt.

decoding	agreement per level	identical prefix
greedy	1.000, 1.000, 1.000, 1.000	562 of 562
top-k, seed 1	0.996, 1.000, 1.000, 0.996	26
top-k, seed 2	1.000, 0.998, 0.993, 0.998	26

The model can reproduce audio exactly, and when it does, it sounds exactly like the tokenizer's reconstruction. Better sound requires finer tokens, and finer tokens are where our next experiments go.

9.2 Lyrics

We extended both vocal models with a lyrics field and trained them on vocal songs, JamendoLyrics and songs whose captions mention rock or a related style. Table 5 gives **LYRICS-TEST** for a model trained on the Thor with the lyrics corpus weighted 8-fold, over 48 batches of 4 windows.

Table 5: Loss with the right, no and wrong lyrics, and $\bar{\Delta} \pm s$.

songs	right	none	wrong	$\bar{\Delta}$
held out (6)	4.3345	4.3409	4.3345	$+0.0000 \pm 0.0004$
training (73)	4.0337	4.0395	4.0378	$+0.0042 \pm 0.0006$

On held-out songs, right and wrong lyrics give the same loss: this model does not yet use the words. On training songs the difference is significant but small, a sign of memorised songs rather than a learned relation between words and sound. The test thus gives a sharp, falsifiable criterion that a future lyrics model must pass.

10 Limitations

The tokenizer limits fidelity, and Figure 7 is the yardstick for any replacement. Griffin-Lim adds phase artefacts that a learned vocoder would remove. The data contain little singing (298 of 706 songs mention a voice) and almost nothing of some styles (2 songs are described as grunge). The model is far smaller than published systems [3], no formal listening test has been carried out, the kernels use 32-bit floats without tensor cores, and attention stores its full score matrix [5].

11 Conclusion

StaleyNET is a complete text-to-music system, from resampling to GPU kernels, written in one language and trained from scratch on edge devices. Owning every stage turned the vague complaint that the output sounds blurred into a precise measurement: the language model reproduces memorised audio exactly, and the tokenizer is the stage that discards the detail a voice needs. The stage-by-stage reconstruction and the paired lyrics test give the project exact targets, and they generalise to any system that pairs a tokenizer with a language model.

References

[1] Andrea Agostinelli et al. MusicLM: Generating music from text, 2023. arXiv:2301.11325.

[2] Zalán Borsos et al. AudioLM: A language modeling approach to audio generation. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2023. arXiv:2209.03143.

[3] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez. Simple and controllable music generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.05284.

[4] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 4 edition, 2022.

[5] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2205.14135.

[6] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi. High fidelity neural audio compression. *Transactions on Machine Learning Research*, 2023. arXiv:2210.13438.

[7] Prafulla Dhariwal, Heewoo Jun, Christine Payne, Jong Wook Kim, Alec Radford, and Ilya Sutskever. Jukebox: A generative model for music, 2020. arXiv:2005.00341.

[8] Simon Durand, Daniel Stoller, and Sebastian Ewert. Similarity-based audio-lyrics alignment of multiple languages. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2023. arXiv:2306.07744.

[9] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *Annual Meeting of the Association for Computational Linguistics*, 2018. arXiv:1805.04833.

[10] Daniel W. Griffin and Jae S. Lim. Signal estimation from modified short-time Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 32(2):236–243, 1984.

[11] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs), 2016. arXiv:1606.08415.

[12] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021. arXiv:2207.12598.

[13] David B. Kirk, Wen mei W. Hwu, and Izzat El Hajj. *Programming Massively Parallel Processors*. Morgan Kaufmann, 4 edition, 2022.

[14] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. HiFi-GAN: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2010.05646.

[15] Rithesh Kumar, Prem Seetharaman, Alejandro Luebs, Ishaan Kumar, and Kundan Kumar. High-fidelity audio compression with improved RVQGAN. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.06546.

[16] Jinglin Liu, Chengxi Li, Yi Ren, Feiyang Chen, and Zhou Zhao. DiffSinger: Singing voice synthesis via shallow diffusion mechanism. In *AAAI Conference on Artificial Intelligence*, 2022. arXiv:2105.02446.

[17] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[18] Ilaria Manco, Benno Weck, SeungHeon Doh, Minz Won, Yixiao Zhang, Dmitry Bogdanov, Yusong Wu, Ke Chen, Philip Tovstogan, Emmanouil Benetos, Elio Quinton, György Fazekas, and Juhan Nam. The song describer dataset: a corpus of audio captions for music-and-language evaluation. In *NeurIPS Workshop on Machine Learning for Audio*, 2023. arXiv:2311.10057.

[19] NVIDIA. cuda-oxide: A Rust to PTX compiler backend. https://github.com/NVlabs/cuda-oxide, 2026.

[20] Arpan Pathak. An introduction to tiled parallel programming, simplified, 2026. Manuscript.

[21] Nathanaël Perraudin, Peter Balazs, and Peter L. Søndergaard. A fast Griffin-Lim algorithm. In *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 2013.

[22] Reiner Pope et al. Efficiently scaling transformer inference. In *Conference on Machine Learning and Systems*, 2023. arXiv:2211.05102.

[23] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[24] Hubert Siuzdak. Vocos: Closing the gap between time-domain and Fourier-based neural vocoders for high-quality audio synthesis, 2023. arXiv:2306.00814.

[25] Malcolm Slaney. Auditory toolbox, version 2. Technical Report 1998-010, Interval Research Corporation, 1998.

[26] Julius O. Smith. Digital audio resampling home page. Center for Computer Research in Music and Acoustics, Stanford University, 2002. https://ccrma.stanford.edu/~jos/resample/.

[27] Daniel Stoller, Simon Durand, and Sebastian Ewert. End-to-end lyrics alignment for polyphonic music using an audio-to-character recognition model. In *IEEE International Conference on Acoustics, Speech and Signal Processing*, 2019. arXiv:1902.06797.

[28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017. arXiv:1706.03762.

[29] Vasily Volkov and James W. Demmel. Benchmarking GPUs to tune dense linear algebra. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2008.

[30] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*, 2020. arXiv:2002.04745.

[31] Linting Xue, Aditya Barua, Noah Constant, Rami Al-Rfou, Sharan Narang, Mihir Kale, Adam Roberts, and Colin Raffel. ByT5: Towards a token-free future with pre-trained byte-to-byte models. *Transactions of the Association for Computational Linguistics*, 2022. arXiv:2105.13626.

[32] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi. SoundStream: An end-to-end neural audio codec. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 2021. arXiv:2107.03312.

[33] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, 2019. arXiv:1910.07467.